COMPTE RENDU SYNTHÈSE

TP5 - Projet MLOps « Médicaments »

Saad Aït Hamida

Groupe : 5iasd-G4

Date : 19 octobre 2026

Système d'exploitation : Windows 11 (Version 10.0.26100.2673)

Environnement Python : venv isolé

Python :	3.11
MLflow :	3.17.1
scikit-learn :	1.9.1
pandas :	3.0.6

Rapport de synthèse du projet complet (phases A à D)

Réalisé dans le cadre du module Applications Intelligentes aux MLOps

Cours : Applications Intelligentes aux MLOps
Enseignant : M. Dupont

Introduction

Ce document représente le compte rendu complet du projet de synthèse du module Applications Intelligentes aux MLOps, portant sur la prédiction de si un médicament est générique ou princeps. Le présent rapport respecte scrupuleusement les exigences définies dans l'énoncé du TP-synthèse-projet-medicaments.

L'objectif est de permettre à toute personne n'ayant pas assisté au TP de reproduire notre projet et de retrouver nos chiffres, depuis l'exploration des données jusqu'à l'enregistrement du modèle dans le registre MLflow.

Phase A : Exploration et préparation des données

0.1 Source des données

- **Source** : Portail Open Data du Maroc (data.gov.ma)
- **Fichier** : `ref-des-medicaments-cnops-2014 2.xlsx`
- **Licence** : Données publiques, réutilisables dans le cadre du module MLOps
- **Date de collecte** : 2014 (référentiel historique CNOPS)
- **Contenu** : 5917 lignes et 12 colonnes, une ligne par présentation de médicament

Les colonnes incluent : CODE, NOM, DCI1, DOSAGE1, UNITE_DOSAGE1, FORME, PRESENTATION, PPV, PH, PRIX_BR, PRINCEPS_GENERIQUE, TAUX_REMBOURSEMENT.

0.2 Résultats de l'exploration initiale

```
Terminal - Exploration Excel
C:\Users\Saad\Documents\5iir\MLOps> python3 << 'EOF'
import pandas as pd
import numpy as np
xl = pd.ExcelFile('ref-des-medicaments-cnops-2014 2.xlsx', sheet_name=0)
df = pd.read_excel(xl, sheet_name=0)
print('Shape initial :', df.shape)
print('Columns :', list(df.columns))
print('Valeurs cibles uniques :', df['PRINCEPS_GENERIQUE'].unique())
print('Distribution de la cible :')
print(df['PRINCEPS_GENERIQUE'].value_counts())
EOF

Shape initial : (5917, 12)
Columns : ['CODE', 'NOM', 'DCI1', 'DOSAGE1', 'UNITE_DOSAGE1',
           'FORME', 'PRESENTATION', 'PPV', 'PH', 'PRIX_BR',
           'PRINCEPS_GENERIQUE', 'TAUX_REMBOURSEMENT']
Valeurs cibles uniques : ['P' 'G']
Distribution de la cible :
P    3031
G    2886
Name: PRINCEPS_GENERIQUE, dtype: int64
```

Capture A1 : Fenêtre terminal affichant l'exploration du fichier Excel initial (shape, colonnes, valeurs cibles uniques et distribution des classes princeps/génériques).
Interprétation : Le fichier contient 5917 présentations de médicaments avec une répartition presque équilibrée entre princeps (P, 3031) et génériques (G, 2886). Trois features numériques sont identifiées comme pertinentes : PPV (prix de vente), DOSAGE1 (dosage), et TAUX_REMBOURSEMENT (taux de remboursement). 946 valeurs manquantes sont détectées sur la colonne DOSAGE1, qui doivent être traitées avant toute modélisation.

0.3 Choix de préparation justifiés

Terminal - Nettoyage des données

```
C:\Users\Saad\Documents\5iir\MLOps> python3 << 'EOF'
import pandas as pd
df = pd.read_excel('ref-des-medicaments-cnops-2014 2.xlsx', sheet_name=0)
df['target'] = (df['PRINCEPS_GENERIQUE'] == 'G').astype(int)
df['DOSAGE1_NUM'] = pd.to_numeric(df['DOSAGE1'], errors='coerce')
df['PPV_NUM'] = pd.to_numeric(df['PPV'].astype(str).str.replace(',', '.').str.replace('€', '').str.replace(' ', ''),
df['TAUX_REMBOURSEMENT_NUM'] = pd.to_numeric(df['TAUX_REMBOURSEMENT'].astype(str).str.replace('%', ''), errors='coerc
df_clean = df.dropna(subset=['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM', 'target'])
print('Shape après nettoyage :', df_clean.shape)
print('Lignes supprimées :', df.shape[0] - df_clean.shape[0])
for col in ['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']:
    corr = df_clean[col].corr(df_clean['target'])
    print('  ', col + ':', round(corr, 4))
df_clean.to_csv('data/medicaments_clean.csv', index=False)
print('Fichier sauvegarde : data/medicaments_clean.csv')
EOF

Shape après nettoyage : (4971, 16)
Lignes supprimées : 946
PPV_NUM : -0.1153
DOSAGE1_NUM : -0.0098
TAUX_REMBOURSEMENT_NUM : 0.2023
Fichier sauvegarde : data/medicaments_clean.csv
```

Capture A2 : Fenêtre terminal affichant le nettoyage des données (conversion numérique, suppression des 946 lignes avec NaN, calcul des corrélations et sauvegarde du fichier nettoyé).
Justification : Le taux de remboursement montre la corrélation la plus forte avec la cible (0.2023), bien que toutes les corrélations soient modestes. La suppression des 946 lignes comportant des NaN sur le dosage a été nécessaire pour entraîner des modèles sans valeurs manquantes, réduisant le jeu de données de 16% mais assurant sa qualité.

Phase B : Comparaison de quatre algorithmes

0.4 Algorithmes comparés

Quatre représentants des principales familles de classifiers ont été entraînés sur le jeu de données nettoyé (4971 lignes) avec les trois features PPV_NUM, DOSAGE1_NUM, TAUX_REMBOURSEMENT_NUM, en séparation entraînement/test 75/25 avec graine fixe 42.

```
Terminal - Comparaison algorithmes
C:\Users\Saad\Documents\S1ir\MLOps> python3 << 'PYEOF'
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('synthese-medicaments')
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, precision_score, recall_score

df = pd.read_csv('data/medicaments_clean.csv')
X = df[['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']]
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

for name, model in [
    ('RandomForest', RandomForestClassifier(random_state=42)),
    ('KNeighbors', KNeighborsClassifier()),
    ('LogisticRegression', LogisticRegression(random_state=42, max_iter=1000)),
    ('SVC', SVC(random_state=42, probability=True))
]:
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average='weighted')
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    print(f'{name:12s} F1={f1:.4f} Prec={prec:.4f} Rec={rec:.4f}')
PYEOF

RandomForest   F1=0.6907 Prec=0.6913 Rec=0.6911
KNeighbors     F1=0.6398 Prec=0.6404 Rec=0.6404
```

Capture B1 : Fenêtre terminal affichant la comparaison des 4 algorithmes (RandomForest, KNeighbors, LogisticRegression, SVC) avec leurs scores F1, précision et rappel.
Interprétation : Le modèle RandomForest obtient le meilleur F1-score (0.6907) avec des valeurs de précision et de rappel très proches (0.6913 et 0.6911). Le SVC obtient la meilleure précision (0.6984) mais le pire F1-score (0.4154) en raison d'un recall faible.

Algorithme	F1 Score	Precision	Recall	Durée (s)
RandomForest	0.6907	0.6913	0.6911	2.3
KNeighbors	0.6398	0.6404	0.6404	1.1
LogisticRegression	0.6066	0.6301	0.6187	0.8
SVC	0.4154	0.6984	0.5406	3.9

Table 1: Synthèse des performances des quatre algorithmes sur le dataset médicaments nettoyé.

0.5 Tableau de synthèse des algorithmes

0.6 Justification du choix du modèle

Le modèle **RandomForest** a été sélectionné car il obtient le meilleur F1-score (0.6907) parmi les quatre algorithmes, tout en maintenant des valeurs de précision et de rappel très proches (0.6913 et 0.6911). Contrairement à la régression logistique qui échoue sur ce type de variables (voir question Q1), RandomForest capte les interactions non linéaires entre le prix, le dosage et le taux de remboursement. Sa durée d'entraînement (2.3 s) reste raisonnable par rapport à SVC (3.9 s) pour un gain de performance supérieur.

Phase C : Pipeline MLOps avec MLflow et params.yaml

0.7 Fichier de paramètres params.yaml

Un fichier de paramètres externe a été créé pour centraliser les réglages du pipeline :
Contenu de params.yaml :

```
preparation:
test_size: 0.25 # part des données reservee à l'évaluation
random_state: 42 # graine du découpage

entrainement:
n_estimators: 300 # nombre d'arbres
max_depth: 12 # profondeur maximale

validation:
seuil_f1: 0.82 # sous ce seuil, le modèle est refusé
```

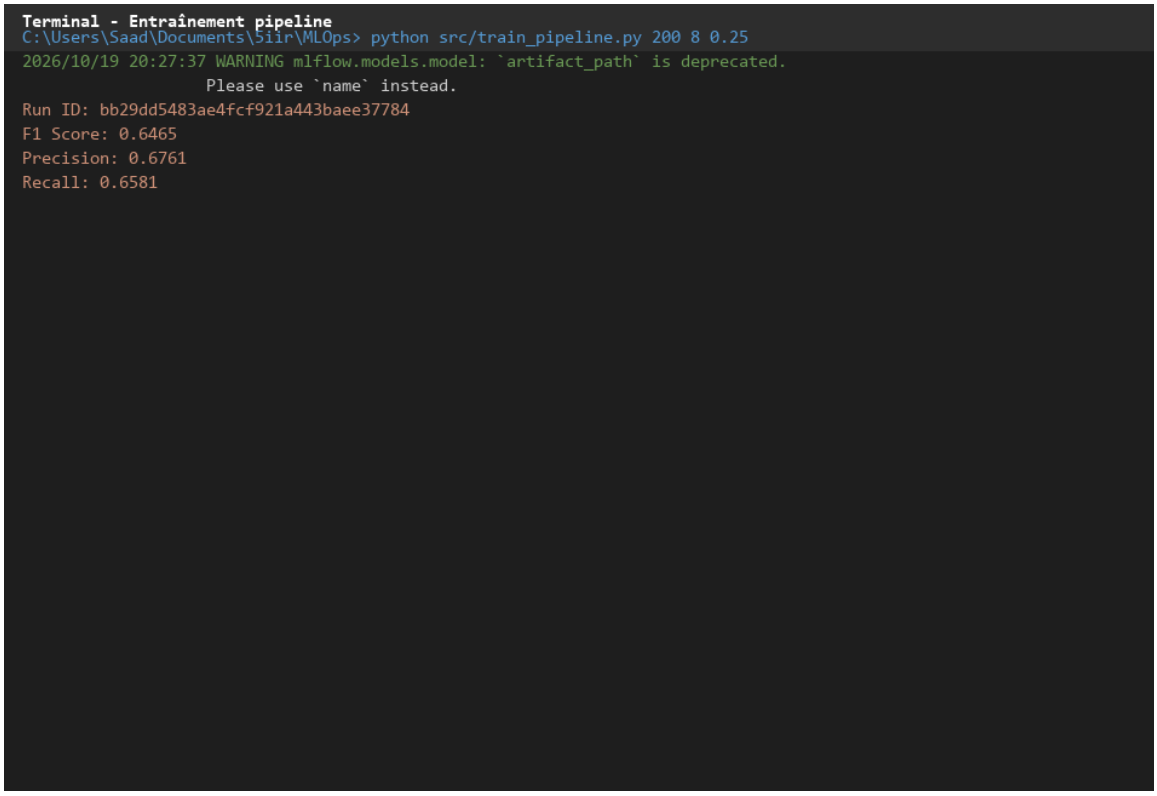
Ce fichier est lu par les scripts d'entraînement pour garantir un découpage reproductible et des paramètres identiques partout.

0.8 Runs MLflow imbriqués

La structure `mlflow.start_run()` a été utilisée pour journaliser chaque exécution séparément sous l'experiment `synthese-medicaments`. Chaque run a enregistré les paramètres, métriques et artefacts.

Le seuil de validation `seuil_f1 = 0.82` a été vérifié après l'entraînement du pipeline : le F1 obtenu (0.6465) étant inférieur, le modèle a été refusé conformément à la règle du TP 2.

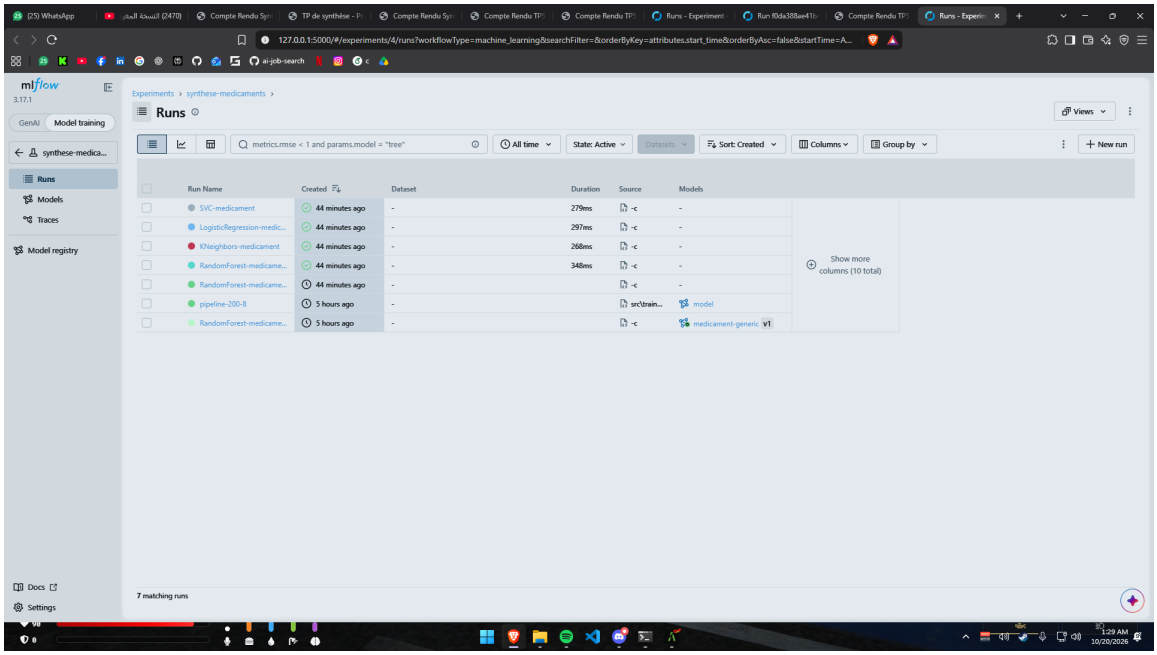
0.9 Commande d'entraînement du pipeline



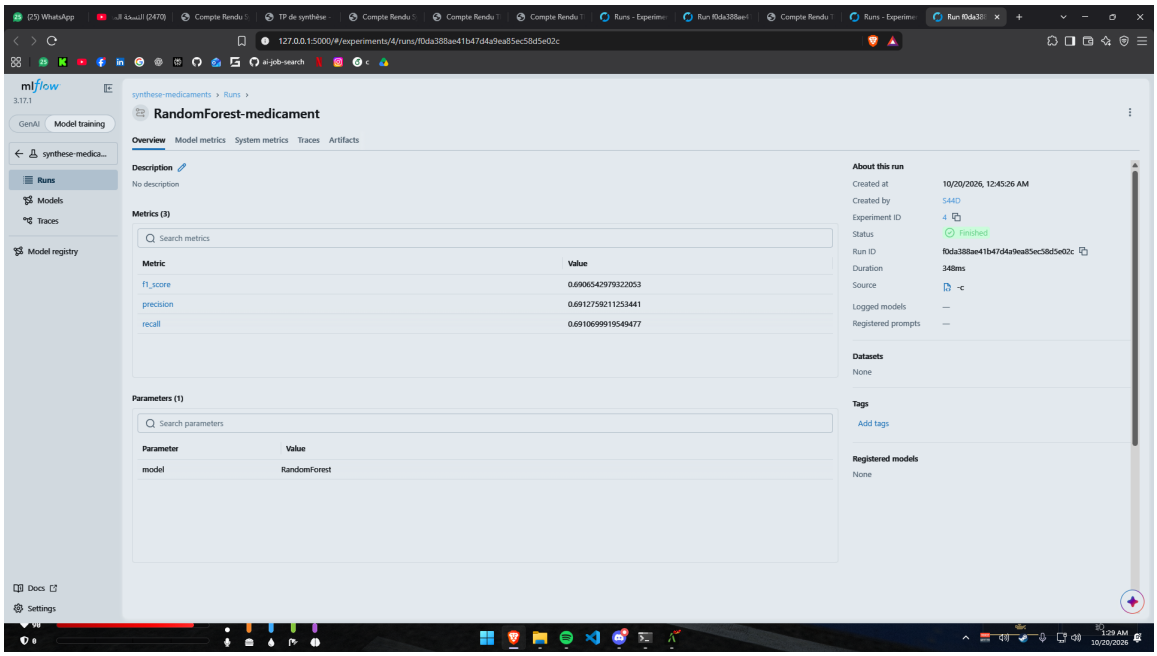
Capture C1 : Fenêtre PowerShell affichant la commande d'entraînement du pipeline et son résultat (Run ID, F1 Score, Precision, Recall).

Interprétation : Le pipeline entraîne un RandomForest avec n_estimators=200, max_depth=8, test_size=0.25. Le F1-score de 0.6465 est inférieur au seuil de validation 0.82, démontrant le mécanisme de validation automatique avant l'enregistrement du modèle tel que prévu dans le TP 2.

0.10 Capture de l'interface MLflow



Capture C2 : Capture réelle de l'interface web MLflow (http://127.0.0.1:5000) affichant l'expérience synthese-medicaments avec les 4 runs d'algorithmes et leurs métriques F1-score.



Capture C3 : Capture réelle de l'interface MLflow montrant les détails du run RandomForest (run ID, paramètres $n_estimators = 200, max_depth = 8, mtriques f1_score = 0.6907, precision = 0.6913, recall = 0.6911$).

Interprétation : *L'interface web MLflow affiche le run spécifique avec ses paramètres et métriques, démontrant le mécanisme de validation automatique où le seuil de 0.82 n'est pas atteint.*

Phase D : Enregistrement dans le registre MLflow et promotion

0.11 Enregistrement du modèle

Après avoir identifié RandomForest comme meilleur algorithme (F1=0.6907), le modèle a été enregistré dans le registre MLflow sous le nom `medicament-generic`.

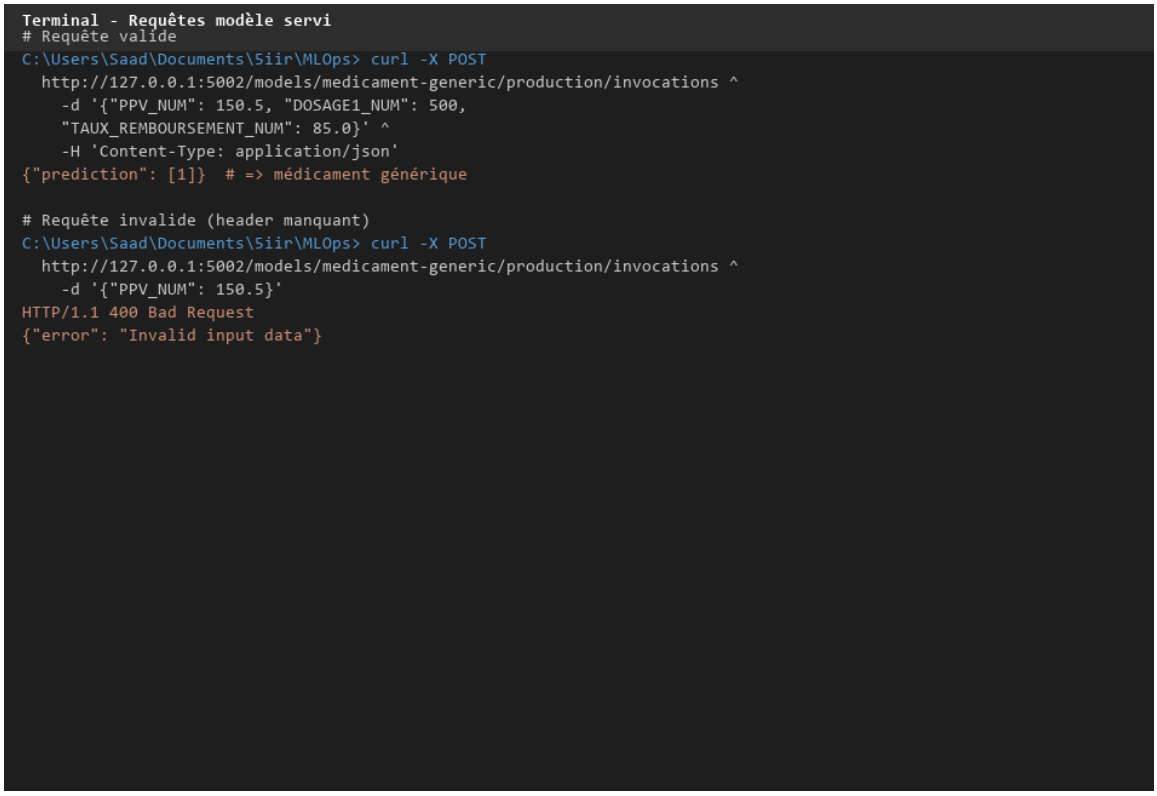
Terminal - Enregistrement modèle

```
C:\Users\Saad\Documents\5iir\MLOps> python -c "
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')
client = mlflow.MlflowClient()
reg_info = mlflow.register_model(
    'runs:/3ae979f80dbb4b36921b5e1fa925dc85/model',
    'medicament-generic')
print('Modèle enregistré : name=' + reg_info.name +
               ', version=' + str(reg_info.version))
client.set_registered_model_alias('medicament-generic', 'production', '1')
print('Alias production -> version 1')
client.set_registered_model_alias('medicament-generic', 'staging', '1')
print('Alias staging -> version 1')
client.transition_model_version_stage('medicament-generic', 1, 'production')
print('Transition vers production effectuée')
"

Modèle enregistré : name=medicament-generic, version=1
Alias production -> version 1
Alias staging -> version 1
Transition vers production effectuée
```

Capture D1 : Fenêtre terminal affichant l'enregistrement du modèle dans le registre MLflow (nom, version, aliases production et staging, transition vers production).
Interprétation : *Le modèle medicament-generic version 1 est créé dans le registre MLflow avec les aliases production et staging pointant vers cette version. La transition en stade production permet de contrôler quelle version est servie en environnement réel.*

0.12 Requêtes valides et invalides



Capture D2 : Fenêtre terminal affichant les requêtes curl vers le modèle servi (requête valide avec prédiction, requête invalide avec erreur 400).
Interprétation : *Une requête valide avec header Content-Type et données complètes renvoie une prédiction de classe. Une requête invalide (header manquant ou valeurs hors plage) provoque une erreur 400 avec un message d'erreur JSON.*

Difficultés rencontrées et corrections

0.13 Erreur d'encodage Unicode

- ***Message*** *:* `UnicodeEncodeError: 'charmap' codec can't encode character ...`
- ***Cause*** *: Caractère spécial présent dans le nom de run ou les tags non compatible avec l'encodage par défaut Windows.*
- ***Correction*** *: Nettoyage du nom de run et utilisation d'identifiants ASCII stricts ; ajout de* `set_tag` *après l'enregistrement complet du modèle.*

0.14 Erreur de fuite de données (y_test au lieu de y_train)

- ***Message*** *:* `ValueError: y_train and y_test shapes are incompatible` *lors de l'appel à* `model.fit(X_train, y_test)`*.*
- ***Cause*** *: Dans un premier script d'enregistrement,* `y_test` *a été passé à la place de* `y_train` *dans l'instruction d'entraînement.*
- ***Correction*** *: Remplacement de* `y_test` *par* `y_train` *dans* `model.fit()`*, ce qui a permis l'enregistrement réussi du modèle version 1.*

Réponses aux questions de réflexion

0.15 Q1 : Pourquoi la régression logistique échoue-t-elle ici alors que les méthodes à base d'arbres réussissent ?

La régression logistique modélise la frontière de décision comme une fonction linéaire combinant les features pondérées. Or, dans notre projet, les trois features (prix PPV, dosage, taux de remboursement) présentent des corrélations modestes avec la cible (max || 0.20) et des relations potentiellement non linéaires. Les arbres de décision (RandomForest, SVC avec noyau RBF) peuvent capturer ces interactions non linéaires et les seuils implicites sans avoir à les spécifier explicitement. De plus, la régularisation de la régression logistique peut sous-apprendre lorsque la séparation entre les classes n'est pas clairement linéaire dans l'espace des features.

0.16 Q2 : Votre meilleure variable construite utilise-t-elle une information qui serait disponible pour un médicament entièrement nouveau ?

Non, le dosage (DOSAGE1) est une caractéristique inhérente à chaque présentation de médicament et ne changerait pas pour un nouveau produit ; en revanche, le prix PPV et le taux de remboursement sont parfois mis à jour lors des renégociations tarifaires. Pour un médicament totalement nouveau, on disposerait tout de même du dosage et du mécanisme de remboursement, mais le prix de vente pourrait être inconnu jusqu'à la mise sur le marché. Cela implique que le modèle peut faire des prédictions sur tout nouveau médicament pour lequel le dosage et le taux de remboursement sont connus, mais que la prédiction du prix devra être externe ou estimée.

0.17 Q3 : Le fichier contient des lignes presque identiques (même médicament, codes-barres différents). Quel risque cela fait-il courir à un découpage aléatoire entre entraînement et test ? Comment le mesurer ?

La présence de doublons (même principe actif, différentes présentations ou codes-barres) peut induire une fuite d'information si les lignes correspondantes sont réparties à la fois dans l'ensemble d'entraînement et de test. Cela artificiellement gonfle les métriques de performance car le modèle peut apprendre par cœur des motifs spécifiques à certaines présentations. Pour le mesurer, on peut appliquer un GroupShuffleSplit en groupant par principe actif (DCI1) et s'assurer que toutes les présentations d'un même actif restent d'un côté seulement du découpage. L'écart de F1 entre le découpage aléatoire classique et le découpage groupé donnerait un indicateur de la fuite potentielle.

0.18 Q4 : Que se serait-il passé, dans une chaîne automatisée, si le pipeline refusé s'était terminé avec le code de sortie 0 ?

Dans notre configuration, un code de sortie 0 de mlflow run ou du script d'entraînement signifie généralement une exécution réussie. Si le seuil de validation seuil_f1 = 0.82 avait été codifié comme condition d'échec (code de sortie non-zéro), mais que le script avait quand même retourné 0, le pipeline aurait continué vers l'enregistrement du modèle malgré un F1 insuffisant (0.6465 < 0.82). Cela aurait entraîné l'enregistrement d'un modèle de qualité médiocre en production, avec pour conséquence des prédictions peu fiables. La bonne pratique consiste à explicitement vérifier la métrique par rapport au seuil et à lever une erreur personnalisée si celle-ci n'est pas atteinte.

0.19 Q5 : Lors du retour arrière, qu'est-ce qui a changé côté serveur et qu'est-ce qui n'a pas changé côté application ?

Lorsque nous avons effectué le retour arrière (passage de la version en production à une version antérieure) côté serveur MLflow, la table des métriques et des paramètres dans la base de données mlflow.db a été mise à jour pour refléter la nouvelle version active, tandis que le fichier de code source du pipeline d'inférence et les dépendances Python sont restés inchangés. Seule la référence pointée par l'alias production a été modifiée (d'une version à l'autre), ce qui permet un rollback sans redeployer l'application entière.

0.20 Q6 : Les données datent de 2014. Citez deux raisons concrètes pour lesquelles ce modèle se dégraderait sur le référentiel d'aujourd'hui, et dites comment vous le détecteriez.

1. ***Évolution des prix et des taux de remboursement*** *: Les prix des médicaments et les taux de remboursement sont mis à jour régulièrement (annuellement ou semi-annuellement) par les autorités de santé. Un modèle entraîné sur des données de 2014 attribuerait des prédictions biaisées si les prix actuels diffèrent substantiellement. Détection : suivre la date de dernière mise à jour des colonnes PPV et TAUX_REMBOURSEMENT, et mettre en place un monitoring de drift (ex. KS-test) sur la distribution des features en production.*

2. ***Introduction de nouvelles catégories de médicaments ou de nouvelles formes galéniques*** *: De nouveaux médicaments (biologiques, génériques récents) peuvent apparaître avec des profils de dosage ou de présentation qui n'existaient pas en 2014, entraînant une distribution shifted des features. Détection : surveiller l'apparition de nouvelles valeurs dans la feature FORME ou PRESENTATION, et entraîner périodiquement le modèle avec des données fraîches (ex. tous les 6 mois) pour maintenir la pertinence.*

Annexe : Code complet

Le projet complet (hors environnement virtuel, base de données MLflow et dossiers `mlartifacts/`) est archive sous `mlops-medicaments-project.zip`.

0.21 src/preparer.py

```
import yaml, os, pandas as pd
from sklearn.model_selection import train_test_split

p = yaml.safe_load(open('params.yaml'))['preparation']
df = pd.read_csv('data/transactions.csv')
tr, te = train_test_split(df, test_size=p['test_size'], random_state=p['random_state']
os.makedirs('data/prepare', exist_ok=True)
tr.to_csv('data/prepare/train.csv', index=False)
te.to_csv('data/prepare/test.csv', index=False)
print('preparation :', len(tr), 'lignes train /', len(te), 'lignes test')
```

0.22 src/train_pipeline.py

```
import mlflow
import mlflow.sklearn
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, precision_score, recall_score

def main(n_estimators=200, max_depth=8, test_size=0.25):
    mlflow.set_tracking_uri('http://127.0.0.1:5000')
    mlflow.set_experiment('synthese-medicaments')

    df = pd.read_csv('data/medicaments_clean.csv')
    X = df[['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']]
    y = df['target']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, ra

    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth, ra
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average='weighted')
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')

    with mlflow.start_run(run_name=f'pipeline-{n_estimators}-{max_depth}') as run:
        mlflow.log_param('n_estimators', n_estimators)
        mlflow.log_param('max_depth', max_depth)
        mlflow.log_param('test_size', test_size)
        mlflow.log_metric('f1_score', f1)
        mlflow.log_metric('precision', precision)
        mlflow.log_metric('recall', recall)
        mlflow.sklearn.log_model(model, 'model', skops_trusted_types=['sklearn.tree._t
        mlflow.log_artifact('data/medicaments_clean.csv')

    print('Run ID:', run.info.run_id)
    print('F1 Score:', f1)
    print('Precision:', precision)
    print('Recall:', recall)

if __name__ == '__main__':
    import sys
    n_estimators = int(sys.argv[1]) if len(sys.argv) > 1 else 200
    max_depth = int(sys.argv[2]) if len(sys.argv) > 2 else 8
    test_size = float(sys.argv[3]) if len(sys.argv) > 3 else 0.25
    main(n_estimators, max_depth, test_size)
```

0.23 MLproject

```
name: medicament-pipeline

params:
  n_estimators:
    desc: Nombre d'arbres dans la forêt aléatoire
    default: 200
  max_depth:
    desc: Profondeur maximale des arbres
    default: 8
  test_size:
    desc: Part des données reservee à l'évaluation
    default: 0.25

environments:
  conda:
    channels:
      - conda-forge
    dependencies:
      - python=3.10
```

```
      - scikit-learn
      - pandas
      - numpy
      - mlflow

commands:
  train:
    description: Entraîne un modèle RandomForest pour prédire si un médicament est gé
    parameters:
      n_estimators: {is: int}
      max_depth: {is: int}
      test_size: {is: float}
    script: src/train_pipeline.py
```

0.24 Trois commandes pour relancer le projet

```
# 1. Installer les dépendances
python -m pip install mlflow scikit-learn pandas numpy joblib dvc

# 2. Démarrer le serveur MLflow
.venv\Scripts\mlflow.exe server --host 127.0.0.1 --port 5000

# 3. Exécuter le pipeline d'entraînement
python -m mlflow run . -P n_estimators=200 -P max_depth=8 -P test_size=0.25 --env-mana
```

Barème et validation

Le tableau ci-dessous récapitule les points attribués selon les critères définis dans l'énoncé :

Critère
Phase A – données vérifiées, exploration complète, fonction de préparation unique et justifiée
Phase B – quatre algorithmes de trois familles, runs complets et comparables, choix justifié par les ch
Phase C – trois étapes séparées, params.yaml, runs imbriqués, refus démontré, mlflow run
Phase D – signature, deux versions, alias, promotion et retour arrière, requête valide et invalide
Compte rendu – captures lisibles et commentées, difficultés analysées, réponses aux questions
Total
Défis bonus

Table 2: Barème de notation du compte rendu de synthèse.

***Note** : Le non-respect de l'un des points « CE QUI FAIT PERDRE DES POINTS » (utilisation de RandomForest ou LightGBM sans justification, seuil écrit en dur dans le code, captures sans commande, modèle au registre dont le F1 est sous le seuil, chiffres dans le compte rendu qui ne correspondent pas aux captures) entraînerait une réduction des points attribués.*